# Does compressing an AI's memory make it forget customer complaints?

## The question

In this study, the memory-saving methods did not clearly lose customer complaints more often than keeping full memory. The pre-set test for "they do" passed for 0 of 3 methods.

## What was done

- **Reviews.** 12 real restaurants from the Yelp dataset, 30 real reviews each. None of the 30 already mentioned the complaints being tested.
- **One planted complaint.** A fake 2-star review, marked as synthetic everywhere, was hidden at review 1, 15 or 30. It described one of three problems: no wheelchair or step-free entrance, a card charged twice, or a car towed from the restaurant's lot. It was padded to a normal review length so it wouldn't stand out.
- **Controls.** Each restaurant also got one set with no planted complaint, to catch the model inventing complaints. That gives 48 sets in total: 36 planted and 12 controls.
- **Five ways to run the model.** "Full" keeps all its notes. Four methods throw away 75% and keep 25%: Random, StreamingLLM, SnapKV and Knorm.
- **Two questions per run.** First, "list the 5 most important recommendations for the owner, citing review numbers". Second, a direct yes/no question: "did any customer mention this problem?"
- **Scale.** 48 sets × 5 methods = 240 runs, with Qwen2.5-1.5B-Instruct on a Colab T4 GPU. For every run, a recorder also logged exactly which notes each method kept, so I could measure how much of the planted review survived.

## How a "mention" was judged

The word check was used for every final number, because it matched the labels perfectly.

Two automatic checks were compared. The **word check** looks for words such as "wheelchair", "towed" or "charged twice"; that list was written and frozen before any model output existed. The **review-number check** looks for a citation of the planted review, such as "[Review 15]".

To decide which to trust, 36 answers were labelled with a written rule, shuffled and with their method hidden. I labelled all 36 myself with this rule: 1 if the advice states the specific problem, in any words, or recommends fixing it; 0 if it only gives a review number, names a related but different issue, or is too vague to tell.

| Check | Found correctly | False alarms | Missed |
|---|---|---|---|
| Word check | 5 | 0 | 0 |
| Review-number check | 3 | 5 | 2 |

The review-number check was fooled by answers that simply listed "Review 1, Review 2, ... Review 30". The detector was chosen after labelling, by the best combined score (F1). The word check wins under any common rule, but only 5 answers in the sample really mentioned the complaint, so this comparison rests on few cases.

## What I found

- **Even with full memory, the model rarely mentioned the complaint.** Full mentioned it in only 7 of 36 sets (19%). The model mostly wrote about food and service.
- **No method met the main test.** The test set before the runs: at least 2 of StreamingLLM, SnapKV and Knorm mention the complaint 10 or more points less often than Full, while still writing proper lists. 0 of 3 methods met it. This holds however the "proper lists" condition is read: none of the three dropped by 10 points in the first place (StreamingLLM 8 points, SnapKV 0, and Knorm mentioned it 11 points more often than Full).
- **Random was clearly worse.** It mentioned the complaint 19 points less often than Full overall, and only 23% of its answers were proper lists.
- **Knorm was clearly better at review 1.** When the complaint came first, Knorm mentioned it 58 points more often than Full. This was the surprise of the study.
- **StreamingLLm:** StreamingLLM keeps only the end of the text: it kept almost nothing of the complaint at reviews 1 and 15, and mentioned it only at review 30. SnapKV kept most at review 30, Knorm kept more at review 1, and Random kept about 25% everywhere.
- **Why methods missed the complaint.** StreamingLLM's misses were mostly cases where the complaint's notes were thrown away. Random's misses were mostly cases where scattered fragments were kept but the model couldn't use them.
- **The model rarely invented complaints.** In the 12 sets with nothing planted, complaint words appeared in at most 2 of 36 checks per method, and the yes/no question wrongly said "yes" at most once in 12.
- **Full itself was messy.** Only 54% of Full answers were proper 5-item lists, and 25 of 48 hit the 512-token answer limit, often stuck in loops. SnapKV, Knorm and StreamingLLM managed 67 to 83%.

## How this relates to the two readings:

1. **Even with its full memory, the AI rarely passed on the complaint:** about 1 time in 5. It mostly talked about food and service.
2. **Throwing away 75% of its notes didn’t clearly make that worse** for the three main methods, so your main test wasn’t met.
3. **Random throwing-away broke the AI:** it never mentioned the complaint, and most of its answers were garbled.
4. **Where the complaint sits matters.** StreamingLLM, which keeps only the end, lost the complaint whenever it wasn’t the last review.
5. **The surprise:** Knorm, the maths-score method, mentioned a complaint placed first far more often than full memory did, because it keeps the start and drops much of the rest.

## DISCLAIMERS:

- **No pre-registration was written before the runs.** The rules that came from the task were fixed before any results existed: the 0.2 cut-off for "thrown away", the calibration rule and the main test above. The rule for choosing the detector was decided after labelling.
- **The baseline was low.** Because Full mentioned the complaint only 19% of the time, there was little room for any method to drop by 10 points, so the main test was hard to meet.
- **The sample is small.** 12 restaurants, one small model (1.5B), one compression level (75% thrown away), and only 5 real mentions in the labelled sample.
- **The answer limit cut answers short.** At 256 tokens every calibration answer was cut off, so the limit was raised to 512 before the real runs. Even so, 25 of 48 Full answers hit the limit. I did not raise it again, because that would have meant redoing all 240 runs after seeing results.
- **AI assistance.** Claude wrote the pipeline code. I wrote the four tests ( except test_licence.py which needed AI assistance as my code had too many loopholes ) and wrote the word lists (lexicon.py) and the six synthetic complaints (templates.py).
- My own decisions, and Claude's, are listed in `logs/decisions.md`.

## Bottom line

In this setup, throwing away 75% of the model's notes did not reliably make it drop complaints, except when the notes were thrown away at random. Whether a complaint survives depends mostly on where it sits and which method is used, and testing that properly needs more restaurants and more labelled answers.